TIPO: Práctica **CALIFICACIÓN:**

MATERIA: Extracción de conocimientos en Base de Datos **MAESTRO:** M.C. MARTHA CÁRDENAS MACIEL

ALUMNO: Mejia Gonzalez David **GRUPO:** 9 IDGS **FECHA DE APLICACIÓN:**

Practica 2. Pipeline ETL y Limpieza de Datos con Pandas

Instrucciones: Realizar un script de Python utilizando la librería Pandas, para realizar el ETL en el conjunto de datos del archivo pokemon.csv. Documente y evidencie el desarrollo de cada uno de los pasos indicados en cada Fase.

Modalidad: Individual

Objetivo de la Práctica: Desarrollar un flujo de trabajo ETL (Extract, Transform, Load) para extraer los datos crudos del archivo *pokemon.csv*, diagnosticar y corregir problemas de calidad de datos (desplazamientos de columnas, espacios en blanco, tipografía inconsistente y valores nulos/erróneos), e ingestar el conjunto transformado en un formato limpio listo para análisis.

El proceso de Extracción, transformación y carga (ETL) es un proceso de integración de datos que combina, limpia y organiza datos de uno o múltiples fuentes en un único conjunto de datos coherente. Luego carga esos datos en un *data warehouse*, *data lake* u otro sistema de destino.



Ilustración 1. Pipeline ETL Pokemon

Fase 1: Extracción (Extract)

En esta fase se realiza la lectura de la fuente primaria en un **DataFrame de Pandas** y la inspección inicial de metadatos.

1. Carga el *dataset* desde el archivo '*pokemon.csv*' utilizando Pandas.

Cargué el archivo con pd.read_csv().

```
# Paso 1. Cargar el dataset
dataFrame_Original = pd.read_csv(archivo_entrada)
```

2. Haz una copia del *dataframe* y trabaja en ella para mantener los datos crudos (originales).

Hice una copia de trabajo.

```
# Paso 2. Copiar el dataframe
dataFrame_Limpio = dataFrame_Original.copy()
```

3. Verifica del conjunto de datos: estructura, registros totales, dimensiones y tipos de datos de cada columna.
    a. Inspecciona las primeras 5 filas o registros

En las primeras cinco filas ya se ve el patrón general: número, nombre, uno o dos tipos, altura, peso y un 0 o 1 para indicar si es legendario.

```
display(dataFrame_Original.head())
```

✓ 0.0s

	N.°	Nombre	Tipo1	Tipo2	Altura	Peso	Legendario
0	1	Bulbasaur	Planta	Veneno	0.7	6.9	0.0
1	2	Ivysaur	Planta	Veneno	1.0	13.0	0.0
2	3	Venusaur	Planta	Veneno	2.0	100.0	0.0
3	4	Charmander	Fuego	NaN	0.6	8.5	0.0
4	5	Charmeleon	Fuego	NaN	1.1	19.0	0.0

b. Obtén la estructura y tipos de datos del *dataframe*

N.º se lee como texto porque hay valores que no son números puros, como 15 Beedrill y 14 4. Nombre, Tipo1 y Tipo2 también quedan como texto. Altura y Peso sí entran como numéricos, y Legendario entra como flotante (0.0 y 1.0) porque tiene huecos.

```
# Paso 3. Verificar estructura, dimensiones y tipos
print(dataFrame_Original.shape)
print(dataFrame_Original.columns.tolist())
dataFrame_Original.info()
```

[4] ✓ 0.0s

```
(150, 7)
['N.°', 'Nombre', 'Tipo1', 'Tipo2', 'Altura', 'Peso', 'Legendario']
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   N.°         150 non-null    str
 1   Nombre      150 non-null    str
 2   Tipo1       150 non-null    str
 3   Tipo2       72 non-null     str
 4   Altura      150 non-null    float64
 5   Peso        150 non-null    float64
 6   Legendario  144 non-null    float64
dtypes: float64(3), str(4)
memory usage: 8.3 KB
```

c. Obtén los datos estadísticos del dataFrame

La media y los extremos de altura y peso existen, pero varias filas están corridas de columna, así que esos números todavía no describen bien a los Pokémon.

```
# Paso 3c. Estadísticas
display(dataFrame_Original.describe(include="all").T)
```

✓ 0.0s

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
N.°	150	150	1	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Nombre	150	150	Bulbasaur	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Tipo1	150	29	Agua	27	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Tipo2	72	21	Veneno	17	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Altura	150.0	NaN	NaN	NaN	2.423333	10.310791	0.2	0.7	1.1	1.5	122.0
Peso	150.0	NaN	NaN	NaN	44.949333	59.611774	0.0	9.0	30.0	55.85	460.0
Legendario	144.0	NaN	NaN	NaN	0.020833	0.143325	0.0	0.0	0.0	0.0	1.0

d. Cuenta los valores nulos por columna (utiliza **isnull().sum()**)

Muchos Pokémon solo tienen un tipo. Los 6 nulos de Legendario sí vienen del desplazamiento de columnas, porque el valor se salió de su lugar y la última celda quedó vacía.

```
# Paso 3d. Valores nulos
display(dataFrame_Original.isnull().sum().to_frame("Valores nulos"))
```

✓ 0.0s

	Valores nulos
N.°	0
Nombre	0
Tipo1	0
Tipo2	78
Altura	0
Peso	0
Legendario	6

Fase 2: Transform (Transformación y Limpieza de Datos)

Al inspeccionar el *dataset* de se identifican las siguientes acciones de limpieza clave que deben ejecutarse, si identificas otras acciones de limpieza de datos documéntalas en este apartado.

Acciones de limpieza

1. Cambia el índice de la primera columna 'N.º'
2. Corrección de espacios en blanco.
    a. Algunos datos de las columnas *string* contienen espacios en blanco al inicio o al final (ej. ' Bicho', ' Normal', ' Veneno').
    Aplicar **str.strip()** en las columnas *string* para normalizar la representación de cadenas.

Usé N.º como índice para identificar cada registro con su número de Pokédex y no con la posición de la fila.
Apliqué str.strip() a todas las columnas de texto. Sin ese paso, Pandas trataría "Normal" y " Normal" como categorías distintas, aunque para nosotros sean la misma.

```
# Paso 1. Quitar espacios en blanco
columnas_texto = dataFrame_Limpio.select_dtypes(include=["object", "str"]).columns

for columna in columnas_texto:
    dataFrame_Limpio[columna] = dataFrame_Limpio[columna].str.strip()

display(dataFrame_Limpio.head())
```

✓ 0.0s

	N.°	Nombre	Tipo1	Tipo2	Altura	Peso	Legendario
0	1	Bulbasaur	Planta	Veneno	0.7	6.9	0.0
1	2	Ivysaur	Planta	Veneno	1.0	13.0	0.0
2	3	Venusaur	Planta	Veneno	2.0	100.0	0.0
3	4	Charmander	Fuego	NaN	0.6	8.5	0.0
4	5	Charmeleon	Fuego	NaN	1.1	19.0	0.0

3. Reorganización de Registros Desplazados.
   Definir y documentar una estrategia para reasignar los valores correctos en las columnas correspondientes (N.º, Nombre, Tipo1, Tipo2, Altura, Peso, Legendario).
   a. Existen filas donde faltan valores o se concatenaron campos durante el guardado original, lo que provocó un desplazamiento horizontal de columnas.
      Ejemplo:
      Índice 14 (Beedrill): El campo **N.º** contiene '15 Beedrill', desplazando **Tipo1** a **Nombre** y enviando **Peso** a 0.0.
      Pokémon monotipo (Rattata, Raticate, Dratini, Dragonair, Mewtwo): Al omitirse el **Tipo2**, la **Altura** se guardó en la columna **Tipo2**, el **Peso** en **Altura**, y el peso real quedó en 0.0 o desplazó a la columna **Legendario**.

Restauré la fila como número 15, Beedrill, Bicho/Veneno, 1.0 m, 29.5 kg y no legendario.
En los registros: Tipo2,Altura y Legendario puse Ninguno en Tipo2 y devolví altura, peso y legendario a su columna. Mewtwo sí es legendario, así que ahí dejé 1; los otros quedaron en 0.

```python
# Paso 2. Reorganizar registros desplazados
fila_beedrill = dataFrame_Limpio["N.°"].astype(str).str.contains("Beedrill", na=False)

dataFrame_Limpio.loc[
    fila_beedrill,
    ["N.°", "Nombre", "Tipo1", "Tipo2", "Altura", "Peso", "Legendario"]
] = ["15", "Beedrill", "Bicho", "Veneno", 1.0, 29.5, 0]

correcciones_monotipo = {
    "Rattata": (0.3, 3.5, 0),
    "Raticate": (0.7, 18.5, 0),
    "Dratini": (1.8, 3.3, 0),
    "Dragonair": (4.0, 16.5, 0),
    "Mewtwo": (2.0, 122.0, 1),
}

for nombre, (altura_correcta, peso_correcto, legendario_correcto) in correcciones_monotipo.items():
    fila = dataFrame_Limpio["Nombre"].eq(nombre)
    dataFrame_Limpio.loc[fila, "Tipo2"] = "Ninguno"
    dataFrame_Limpio.loc[fila, "Altura"] = altura_correcta
    dataFrame_Limpio.loc[fila, "Peso"] = peso_correcto
    dataFrame_Limpio.loc[fila, "Legendario"] = legendario_correcto

dataFrame_Limpio["N.°"] = dataFrame_Limpio["N.°"].replace({"14 4": "144"})

nombres_revisar = ["Beedrill", "Rattata", "Raticate", "Dratini", "Dragonair", "Mewtwo", "Articuno"]
display(dataFrame_Limpio[dataFrame_Limpio["Nombre"].isin(nombres_revisar)])
```

✓ 0.0s

	N.°	Nombre	Tipo1	Tipo2	Altura	Peso	Legendario
14	15	Beedrill	Bicho	Veneno	1.0	29.5	0.0
18	19	Rattata	Normal	Ninguno	0.3	3.5	0.0
19	20	Raticate	Normal	Ninguno	0.7	18.5	0.0
143	144	Articuno	Hielo	Volador	1.7	55.4	1.0
146	147	Dratini	Dragón	Ninguno	1.8	3.3	0.0
147	148	Dragonair	Dragón	Ninguno	4.0	16.5	0.0
149	150	Mewtwo	Psíquico	Ninguno	2.0	122.0	1.0

4. Estandarización de Categorías y Errores Tipográficos
    a. Existen valores en **Tipo1** y **Tipo2** con sinónimos, erratas o términos en inglés Ejemplo: 'Wa ter', 'Hierba', 'Insecto', 'I ce', 'Veneno n', 'luchando', 'Lucha ng', 'psíquico', 'hada'). Mapear y unificar la taxonomía oficial de tipos:
    cambiar 'Wa ter' a 'Agua', 'Hierba' a 'Planta', 'Insecto' a 'Bicho', 'I ce' a 'Hielo', etc. (Puedes utilizar el método replace())

En Tipo1 y Tipo2 hay sinónimos, errores y cambios de mayúsculas. Los unifiqué con replace() hacia los nombres de tipo que se usan en el resto del archivo:

```python
# Paso 3. Estandarizar categorías
correcciones_tipos = {
    "Wa ter": "Agua",
    "Hierba": "Planta",
    "Insecto": "Bicho",
    "I ce": "Hielo",
    "Veneno n": "Veneno",
    "luchando": "Lucha",
    "Lucha ng": "Lucha",
    "psíquico": "Psíquico",
    "hada": "Hada",
}

for columna in ["Tipo1", "Tipo2"]:
    dataFrame_Limpio[columna] = dataFrame_Limpio[columna].replace(correcciones_tipos)

dataFrame_Limpio["Nombre"] = dataFrame_Limpio["Nombre"].replace({"Bellspr fuera": "Bellsprout"})

print(sorted(dataFrame_Limpio["Tipo1"].dropna().unique()))
print(sorted(dataFrame_Limpio["Tipo2"].dropna().unique()))
```

✓ 0.0s Python

```
['Agua', 'Bicho', 'Dragón', 'Eléctrico', 'Fantasma', 'Fuego', 'Hada', 'Hielo', 'Lucha', 'Normal', 'Planta', 'Psíquico', 'Roca',
['Acero', 'Agua', 'Hada', 'Hielo', 'Lucha', 'Ninguno', 'Planta', 'Psíquico', 'Roca', 'Tierra', 'Veneno', 'Volador']
```

5. Tratamiento de Valores Faltantes (NaNs)
    a. La columna Tipo2 contiene valores nulos legítimos (Pokémon de un solo tipo). Asignar la categoría 'Ninguno' estructurado para Tipo2 cuando aplique. (Puedes utilizar el método fillna())
    b. Legendario presenta nulos debido a los desplazamientos de filas. Colocar los valores de Legendario faltantes con 0 (Utiliza fillna(0)).

Los rellené con fillna("Ninguno") para que la ausencia quede explícita y no se confunda con un dato perdido.

```
# Paso 4. Tratar valores faltantes
dataFrame_Limpio["Tipo2"] = dataFrame_Limpio["Tipo2"].fillna("Ninguno")
dataFrame_Limpio["Legendario"] = dataFrame_Limpio["Legendario"].fillna(0)

display(dataFrame_Limpio.isnull().sum().to_frame("Valores nulos"))
```

✓ 0.0s

	Valores nulos
N.°	0
Nombre	0
Tipo1	0
Tipo2	0
Altura	0
Peso	0
Legendario	0

6. Corrección de errores Numéricos y Conversión de tipos de datos
    a. El campo **N.º** contiene el valor en cadena '14 4' para Articuno en lugar de 144.
       Corregir el ID numérico 144.
       Cambiar la columna **N.º** a numérico entero (utiliza astype(int))
    b. Legendario está almacenado como punto flotante (float64) con valores 0.0 y 1.0
       Cambiar la columna Legendario a tipo booleano (bool).

El identificador de Articuno venía como 14 4. Lo cambié a 144 antes de convertir la columna. Luego pasé N.º a entero con to_numeric y astype(int), y Altura y Peso a numérico. Legendario lo convertí a booleano, así que 1 queda como True y 0 como False.

```python
# Paso 5. Corregir tipos de datos y usar N.° como índice
dataFrame_Limpio["N.°"] = pd.to_numeric(dataFrame_Limpio["N.°"], errors="raise").astype(int)
dataFrame_Limpio["Altura"] = pd.to_numeric(dataFrame_Limpio["Altura"], errors="raise")
dataFrame_Limpio["Peso"] = pd.to_numeric(dataFrame_Limpio["Peso"], errors="raise")
dataFrame_Limpio["Legendario"] = dataFrame_Limpio["Legendario"].astype(bool)

dataFrame_Limpio = dataFrame_Limpio.set_index("N.°")
dataFrame_Limpio = dataFrame_Limpio.reset_index()

display(dataFrame_Limpio.dtypes.to_frame("Tipo de dato"))
```

✓ 0.0s

	Tipo de dato
N.°	int64
Nombre	str
Tipo1	str
Tipo2	str
Altura	float64
Peso	float64
Legendario	bool

Fase 3: Load (Carga)

Exportación del conjunto de datos procesado y limpio a un archivo estructurado para su posterior análisis o modelado.

dataFrame_Limpio.to_csv('pokemon_limpio.csv', index=False)

Exporté el DataFrame limpio.



Fase de Verificación del resultado

Revisar la integridad del *dataFrame* final procesado

Verifica del conjunto de datos: estructura, registros totales, dimensiones y tipos de datos de cada columna.

a. Obtén las primeras 10 filas o registros

```
# Paso a. Primeras 10 filas
display(dataFrame_Limpio.head(10))
```

✓ 0.0s

	N.°	Nombre	Tipo1	Tipo2	Altura	Peso	Legendario
0	1	Bulbasaur	Planta	Veneno	0.7	6.9	False
1	2	Ivysaur	Planta	Veneno	1.0	13.0	False
2	3	Venusaur	Planta	Veneno	2.0	100.0	False
3	4	Charmander	Fuego	Ninguno	0.6	8.5	False
4	5	Charmeleon	Fuego	Ninguno	1.1	19.0	False
5	6	Charizard	Fuego	Volador	1.7	90.5	False
6	7	Squirtle	Agua	Ninguno	0.5	9.0	False
7	8	Wartortle	Agua	Ninguno	1.0	22.5	False
8	9	Blastoise	Agua	Ninguno	1.6	85.5	False
9	10	Caterpie	Bicho	Ninguno	0.3	2.9	False

b. Obtén las últimas 10 filas o registros

```
# Paso b. Últimas 10 filas
display(dataFrame_Limpio.tail(10))
```

✓ 0.0s

	N.°	Nombre	Tipo1	Tipo2	Altura	Peso	Legendario
140	141	Kabutops	Roca	Agua	1.3	40.5	False
141	142	Aerodactyl	Roca	Volador	1.8	59.0	False
142	143	Snorlax	Normal	Ninguno	2.1	460.0	False
143	144	Articuno	Hielo	Volador	1.7	55.4	True
144	145	Zapdos	Eléctrico	Volador	1.6	52.6	True
145	146	Moltres	Fuego	Volador	2.0	60.0	True
146	147	Dratini	Dragón	Ninguno	1.8	3.3	False
147	148	Dragonair	Dragón	Ninguno	4.0	16.5	False
148	149	Dragonite	Dragón	Volador	2.2	210.0	False
149	150	Mewtwo	Psíquico	Ninguno	2.0	122.0	True

c. Obtén 10 registros aleatorios

```
# Paso c. 10 registros aleatorios
display(dataFrame_Limpio.sample(n=min(10, len(dataFrame_Limpio)), random_state=42))
```

✓ 0.0s

	N.°	Nombre	Tipo1	Tipo2	Altura	Peso	Legendario
73	74	Geodude	Roca	Tierra	0.4	20.0	False
18	19	Rattata	Normal	Ninguno	0.3	3.5	False
118	119	Seaking	Agua	Ninguno	1.3	39.0	False
78	79	Slowpoke	Agua	Psíquico	1.2	36.0	False
76	77	Ponyta	Fuego	Ninguno	1.0	30.0	False
31	32	Nidoran♀	Veneno	Ninguno	0.5	9.0	False
64	65	Alakazam	Psíquico	Ninguno	1.5	48.0	False
141	142	Aerodactyl	Roca	Volador	1.8	59.0	False
68	69	Bellsprout	Planta	Veneno	0.7	4.0	False
82	83	Farfetch'd	Normal	Volador	0.8	15.0	False

d. Obtén la estructura y tipos de datos del *dataframe*

```
# Paso d. Estructura y tipos
dataFrame_Limpio.info()
```

✓ 0.0s

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   N.°         150 non-null    int64
 1   Nombre      150 non-null    str
 2   Tipo1       150 non-null    str
 3   Tipo2       150 non-null    str
 4   Altura      150 non-null    float64
 5   Peso        150 non-null    float64
 6   Legendario  150 non-null    bool
dtypes: bool(1), float64(2), int64(1), str(3)
memory usage: 7.3 KB
```

e. Obtén los datos estadísticos del *dataFrame*

```
# Paso e. Estadísticas
display(dataFrame_Limpio.describe(include="all").T)
```
✓ 0.0s

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
N.°	150.0	NaN	NaN	NaN	75.5	43.445368	1.0	38.25	75.5	112.75	150.0
Nombre	150	150	Bulbasaur	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Tipo1	150	15	Agua	28	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Tipo2	150	12	Ninguno	83	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Altura	150.0	NaN	NaN	NaN	1.2	0.963634	0.2	0.7	1.0	1.5	8.8
Peso	150.0	NaN	NaN	NaN	46.231333	59.547388	0.1	9.925	30.0	56.375	460.0
Legendario	150	2	False	146	NaN	NaN	NaN	NaN	NaN	NaN	NaN

f. Cuenta los valores nulos por columna (utiliza **isnull().sum()**)

```
# Paso f. Valores nulos
display(dataFrame_Limpio.isnull().sum().to_frame("Valores nulos"))
print(dataFrame_Limpio.shape)
print(len(dataFrame_Limpio))
```
✓ 0.0s

	Valores nulos
N.°	0
Nombre	0
Tipo1	0
Tipo2	0
Altura	0
Peso	0
Legendario	0

```
(150, 7)
150
```

g. Enlista los datos sin repetir de Tipo1 (Tipos de pokemones)

```
# Paso g. Tipos sin repetir en Tipo1
tipos_unicos = sorted(dataFrame_Limpio["Tipo1"].dropna().unique())
for tipo in tipos_unicos:
    print("-", tipo)

print(len(tipos_unicos))
```

✓ 0.0s

```
- Agua
- Bicho
- Dragón
- Eléctrico
- Fantasma
- Fuego
- Hada
- Hielo
- Lucha
- Normal
- Planta
- Psíquico
- Roca
- Tierra
- Veneno
15
```

¿Qué cambios faltan por realizar en el set de datos resultante y cómo los aplicarías?

Por lo que pude ver en el archivo todavía hay nombres partidos que no corregi:

- Persia n (Persian)
- G olem (Golem)
- Kin gler (Kingler).

El mismo método que usé con Bellsprout puede localizar el registro y aplicar replace() sobre Nombre.

Conclusiones

Con esta práctica me quedó más claro que la limpieza de datos es muy importante para un análisis. La extracción sirvió para ver la forma real del archivo (150 Pokémon, columnas corridas, espacios de más y tipos escritos de varias maneras).

En la fase de transformación tuve que separar lo que es realmente un dato faltante, como el segundo tipo de un Pokémon monotipo, de lo que es un error de captura, como Beedrill o Articuno.

La fase de carga, al final, es la parte corta: exportar pokemon_limpio.csv cuando el DataFrame ya está listo para una revisión de nulos, tipos y categorías.

Referencia de fuentes de información consultada

Pandas Development Team. pandas documentation. https://pandas.pydata.org/docs/